

THEORY ASSIGNMENT

Course: Full Stack Web Development
Course Code: 23CM4121
Topic: JavaScript Functions vs Classes
Student Name: P.Raghu Ram
Roll No.: A24126552102
Date: 30-08-2026

1. QUESTION

Differentiate between JavaScript functions and classes. Explain how a class can be used to create multiple objects with common properties and methods.

2. ANSWER

Difference between JavaScript Functions and Classes

In JavaScript, both functions and classes can be used to construct objects and define behaviors, but they have distinct syntax, features, and conceptual approaches.

Feature	Functions (including Constructor Functions)	Classes (ES6)
Syntax & Nature	Declared using the <i>function</i> keyword. Functions can be standard functions, arrow functions, or constructor functions.	Declared using the <i>class</i> keyword. Introduced in ES6 as syntactic sugar over JavaScript's existing prototype-based inheritance.
Hoisting	Function declarations are hoisted to the top of their scope, meaning they can be called before they are defined.	Class declarations are <i>not</i> hoisted. Attempting to use a class before it is declared results in a <i>ReferenceError</i> .
Constructor	The function itself acts as the constructor when invoked with the <i>new</i> keyword.	Uses an explicit <i>constructor()</i> method within the class body to initialize objects.
Methods & Inheritance	Methods must be manually attached to the function's prototype (e.g., <i>Person.prototype.greet = ...</i>) to be shared.	Methods defined inside a class are automatically added to the prototype. Supports the <i>extends</i> and <i>super</i> keywords for easier inheritance.

Using a Class to Create Multiple Objects

A class serves as a blueprint for creating objects. When a class is defined, it can encapsulate both state (properties) and behavior (methods) that will be shared across all objects (instances) created from that class. This approach is highly efficient because the methods are stored on the prototype, meaning they are not recreated in memory for every single object instance.

Key components when using classes:

- **Blueprint (Class Definition):** Defines the structure.
- **Constructor:** A special method that is automatically called when a new instance is created. It initializes the object's unique properties (e.g., *this.name = name*).
- **Methods:** Functions defined inside the class which are shared by all instances.
- **Instantiation (new keyword):** Used to create a new object instance from the class.

Example Implementation

```
class Student {  
  // Constructor initializes properties specific to each object  
  constructor(name, rollNo) {  
    this.name = name;  
    this.rollNo = rollNo;  
  }  
}
```

```
}

// Method shared by all Student objects
study() {
  console.log(`${this.name} (Roll No: ${this.rollNo}) is studying.`);
}
}

// Creating multiple objects using the 'new' keyword
const student1 = new Student("Raghu Ram", "A24126552102");
const student2 = new Student("Jane Doe", "A24126552103");

// Accessing properties and methods
student1.study(); // Output: Raghu Ram (Roll No: A24126552102) is studying.
student2.study(); // Output: Jane Doe (Roll No: A24126552103) is studying.
```

In the example above, *Student* acts as the blueprint. *student1* and *student2* are distinct objects with their own unique properties (name and rollNo), but they share the exact same *study()* method provided by the class prototype.